

INDEPENDENT ENGINEERING NOTEBOOKS

Cadence

Soft-computing traffic signal control

Author

Repository github.com/Nikhil201716/Cadence-Traffic-Control

Stack

Edition 1.0

How to read this book

Contents

1. What Cadence is

- 1.1 Soft computing, because the inputs are soft
- 1.2 The honest-baseline stance
- 1.3 Soft computing, and where it came from
- 1.4 The five methods, and the part each suits
- 1.5 Tolerance for imprecision

2. The traffic simulator

- 2.1 Delay is the area under the queue
- 2.2 An arterial that carries platoons
- 2.3 The isolated intersection
- 2.4 The honest negative test
- 2.5 Pinning progression in a test
- 2.6 The isolated result, and the modelling choice
- 2.7 Little's law, and why delay is an integral
- 2.8 Saturation flow and startup lost time
- 2.9 The region where the model is trustworthy

3. Fuzzy green time

- 3.1 Fuzzify: membership by degree
- 3.2 Infer and defuzzify
- 3.3 The finding, against a fair baseline
- 3.4 Shoulders, the rule base, and the baseline
- 3.5 Testing the response shape
- 3.6 The fuzzy claim, as a test
- 3.7 A worked inference
- 3.8 The right shoulder, and dispatching by phase
- 3.9 Fuzzy is not probability
- 3.10 Why centroid, and not mean-of-maxima

4. Coordinating the network: GA versus grid search

- 4.1 The objective, and the exhaustive baseline
- 4.2 The genetic algorithm
- 4.3 The result, stated honestly
- 4.4 The GA operators, and the grid
- 4.5 The GA's report, and its cache test
- 4.6 The headline comparison, as a test
- 4.7 What a green wave really is
- 4.8 Fitness caching, and why not annealing
- 4.9 Why tournament selection and uniform crossover
- 4.10 What the exhaustive guarantee is worth
- 4.11 When the heuristic would not help

5. A Hopfield network for noisy detectors

- 5.1 Hebbian storage and asynchronous recall
- 5.2 Recovery, and the capacity cliff
- 5.3 Corruption, and measuring recovery
- 5.4 Testing recall directly
- 5.5 The capacity cliff, as a test
- 5.6 The energy that guarantees convergence
- 5.7 A worked recall, by hand
- 5.8 Spurious states, and the cost of overloading
- 5.9 Asynchronous updating, and why it settles

6. The perceptron, ADALINE, and XOR

- 6.1 Two learning rules
- 6.2 Convergence, and the wall
- 6.3 ADALINE, and the datasets
- 6.4 Testing convergence on separable data
- 6.5 The XOR wall, as a test
- 6.6 Why the delta rule descends smoothly
- 6.7 The two units, and the winter they caused

6.8 The bias, and what a hidden layer buys

6.9 The perceptron convergence theorem

7. Design decisions worth defending

7.1 A queue model, not a car-following one

7.2 Centroid defuzzification, over cheaper alternatives

7.3 Elitism in the genetic algorithm

7.4 Bipolar states in the Hopfield network

7.5 Pinning the first offset

7.6 Seeding everything, including the heuristic

8. How the reports are made

8.1 One writer, one function per experiment

8.2 The offset and soft-computing experiments

8.3 Hopfield, and the whole run

8.4 Why seeds, and why that is not enough

9. Results, and what this does not establish

9.1 What was shown

9.2 What it does not establish

10. How to read these results

10.1 Each win is a specific, bounded claim

10.2 The baseline is the point

10.3 When to reach for each

11. Exercises

11.1 Questions

11.2 Solutions

12. Running the Project, Step by Step

12.1 Prerequisites

12.2 Step 1 - Clone and enter the repository

12.3 Step 2 - Create an isolated environment

12.4 Step 3 - Install dependencies

12.5 Step 4 - Run the pipeline, stage by stage

12.6 Command reference card

12.7 Troubleshooting

13. Appendix

13.1 A - Repository map

13.2 B - Glossary

13.3 C - Reproducibility

13.4 D - Where this leads next

CHAPTER 1

What Cadence is

Traffic control where the inputs are genuinely fuzzy, solved with soft-computing methods written from scratch - and measured against honest baselines, including the exhaustive one a heuristic is supposed to beat.

1.1 Soft computing, because the inputs are soft

A queue is not simply short or long; it is long to a degree. Demand is not on or off; it is heavy to a degree. Waiting is excessive by degree. None of these has a crisp cutoff, and pretending otherwise - a rule that fires at exactly nine vehicles and not eight - throws away the very structure that makes the problem tractable. Cadence takes the inputs as fuzzy and uses the methods built for that: a Mamdani fuzzy inference system for green time, a genetic algorithm for network coordination, a Hopfield network for noisy detectors, and a perceptron and ADALINE for classification.

All four are written from scratch on NumPy alone, because the algorithms are the point - a library call would hide exactly what this project is about. Everything is deterministic and seeded, so every number reproduces.

1.2 The honest-baseline stance

A soft-computing method is only interesting if it beats something simpler, and the comparison only means anything if the baseline is a fair one. So the fuzzy controller is measured against the *best-tuned* fixed split, not an arbitrary one; and the genetic algorithm is measured against *exhaustive grid search*, which finds the true optimum. If the clever method does not beat the baseline, the report says so plainly - the same test a fine-tuned transformer failed in a sibling project, run honestly here too.

WHAT 'WINNING' MEANS HERE

The genetic algorithm does not beat grid search on solution quality - it ties. It wins on time-to-solution, and only decisively as the network grows. Chapter 4 states that distinction rather than blurring it, because a heuristic that merely matches an exhaustive search on a small problem has proven very little.

1.3 Soft computing, and where it came from

The name is Lotfi Zadeh's, who introduced fuzzy sets in 1965 and later grouped fuzzy logic, neural networks and evolutionary computation as "soft computing" - methods that tolerate imprecision and partial truth rather than demanding the crisp certainties of classical logic. Traffic is a natural home for them: the demand is uncertain, the sensors are noisy, the objective is a soft trade between competing

movements, and the search spaces are too large to solve exactly at scale. Each component in Cadence is one of those tools pointed at the part of the problem it suits, and the constraints are the usual ones for this series - reproducible from a seed, one dependency, every number from a report a real run wrote.

1.4 The five methods, and the part each suits

It is worth laying out the whole toolkit before the chapters take each apart, because the point of the project is that different soft-computing methods fit different corners of one problem.

Table 1.1 - four methods, four sub-problems, four honest baselines

Method	The part of the problem it takes	Baseline it is measured against
Mamdani fuzzy system	setting green time from vague queue/demand inputs	the best-tuned fixed split
Genetic algorithm	coordinating offsets across the network	exhaustive grid search
Hopfield network	recovering corrupted detector readings	the stored patterns themselves
Perceptron / ADALINE	classifying a traffic state from features	each other, and the XOR wall

None is a universal tool; each is chosen because the sub-problem has the shape it handles - fuzziness, a combinatorial search, noisy associative recall, or a linear decision. And each is set against a baseline strict enough that beating it means something.

1.5 Tolerance for imprecision

The phrase "soft computing" is Lotfi Zadeh's, and it names a stance more than a toolbox. Hard computing demands precision, certainty and rigour; it wants the exact answer and pays for it in brittleness and cost. Soft computing trades a little of each away on purpose - it tolerates imprecision, uncertainty and partial truth - to buy tractability, robustness and a low solution cost. The four methods in this book are soft in exactly that sense, and the traffic problem rewards the trade at every turn.

- ["The fuzzy controller tolerates imprecision: it acts on "the queue is getting long" rather than demanding a threshold that would be wrong the moment demand shifted.', 'The genetic algorithm tolerates a good-enough answer: it gives up the exhaustive guarantee and buys a solution that scales, which is the only reason it is worth having over grid search.', 'The Hopfield network tolerates corruption: it takes a reading with bits flipped and returns the nearest clean memory, where a lookup table would simply fail to match.', "The neurons tolerate being taught from data rather than programmed, and - in XOR's case - teach us honestly where a single linear unit's tolerance runs out."]

Zadeh's deeper claim was that these methods are complementary, not rivals: a real system blends them, each covering the others' blind spots. Cadence keeps them separate so each can be measured cleanly against its own baseline, which is the book's job. But the seam is visible throughout - the fuzzy

controller could set a genetic algorithm's objective, the Hopfield network could clean the detector feed the fuzzy controller reads - and seeing where the pieces would join is part of understanding why the field groups them under one name.

CHAPTER 2

The traffic simulator

A store-and-forward queue model whose total delay is the integral of queue length over time - the single number every controller is scored against.

2.1 Delay is the area under the queue

Vehicles are modelled as queues, not individual agents. This is the standard store-and-forward model for signal-timing studies: it is deterministic given a seed, and it gives delay a clean definition. By Little's law the total time vehicles spend waiting is the integral of queue length over time, so summing every approach's queue at every second yields total vehicle-seconds of delay.

$$D = \sum_t \sum_a q_a(t)$$

Minimising that sum is the whole objective, whether by choosing green splits at one intersection or offsets across a network.

2.2 An arterial that carries platoons

The network is a one-way arterial of signalised intersections. Each signal shows the main street a green window that starts at its offset within the common cycle; the rest of the cycle serves the cross street. The key coupling is that the arrivals at one intersection are the departures of the previous one, delayed by the travel time - so a platoon released upstream arrives downstream after a fixed delay, and whether it meets green or red is entirely a matter of the offsets.

Listing 2.1 - the arterial simulator

src/cadence/trafficsim.py

```

def simulate_arterial(offsets: list[int], cycle: int, arterial_green: int,
                     travel_time: int, inflow: float, cross_inflow: float,
                     sat_flow: float, horizon: int,
                     lost_time: int = 2) -> ArterialResult:
    """Simulate a one-way arterial of ``len(offsets)`` signalised intersections.

    Signal i shows the arterial a green window of ``arterial_green`` seconds
    starting at ``offsets[i]`` within each ``cycle``; the rest of the cycle
    (minus ``lost_time`` for the amber/all-red) serves the cross street.
    Arterial arrivals at intersection i are intersection (i-1)'s departures
    delayed by ``travel_time``; the first intersection is fed by ``inflow``.
    Cross arrivals are an independent constant demand.

    The only thing the offsets change is when each green window falls, so a
    good offset vector lets a platoon released upstream arrive during the next
    signal's green and pass without stopping - and a bad one makes it stop at
    every signal.
    """
    n = len(offsets)
    arterial_q = [0.0] * n
    cross_q = [0.0] * n
    # departures[i][t] = vehicles that left intersection i's arterial approach at t
    departures = [[0.0] * (horizon + travel_time + 1) for _ in range(n)]

    total_delay = arterial_delay = cross_delay = 0.0
    stops = 0
    served = 0.0

    def arterial_is_green(i: int, t: int) -> bool:
        phase = (t - offsets[i]) % cycle
        return phase < arterial_green

    for t in range(horizon):
        for i in range(n):
            # -- arterial arrivals this second
            if i == 0:
                arr = inflow
            else:
                src = t - travel_time
                arr = departures[i - 1][src] if src >= 0 else 0.0
            arterial_q[i] += arr

            # -- cross arrivals
            cross_q[i] += cross_inflow

            green = arterial_is_green(i, t)
            # lost time at the very start of each green: no discharge yet
            phase = (t - offsets[i]) % cycle
            usable = green and phase >= lost_time

            if usable:
                depart = min(arterial_q[i], sat_flow)
                arterial_q[i] -= depart
                departures[i][t] = depart
                served += depart if i == n - 1 else 0.0
                # a platoon that arrived to a red earlier is a stop; count a
                # stop whenever fresh arrivals could not immediately pass

```

```

        if arr > 0 and arterial_q[i] > arr:
            stops += 1
    else:
        if arr > 0:
            stops += 1
        # cross street discharges when the arterial is red (minus lost)
        cross_phase = phase - arterial_green
        if not green and cross_phase >= lost_time:
            cross_q[i] -= min(cross_q[i], sat_flow)

    arterial_delay += arterial_q[i]
    cross_delay += cross_q[i]

    total_delay = arterial_delay + cross_delay

    return ArterialResult(total_delay, arterial_delay, cross_delay, stops, served)

```

2.3 The isolated intersection

The single-intersection simulator is where the fuzzy controller is tested. Two phases compete; a controller decides how long to hold each green, given the queues; arrivals are seeded, and a startup lost time at the head of each green discharges nothing - drivers reacting and vehicles accelerating - which is the real cost that makes very short greens inefficient.

Listing 2.2 - the isolated-intersection simulator

src/cadence/trafficsim.py

```

def simulate_isolated(controller, demand: list[tuple[float, float]],
                      sat_flow: float, seconds_per_step: int = 1,
                      min_green: int = 5, max_green: int = 60,
                      lost_time: int = 4, seed: int = 0) -> IsolatedResult:
    """One intersection, two competing phases (say NS and EW), a controller that
    decides how long to hold each green.

    ``demand`` is a list of (ns_rate, ew_rate) in vehicles/second, one entry per
    control interval, so the demand can be made asymmetric and time-varying -
    which is exactly where a fixed 50/50 split wastes green on an empty approach
    and an adaptive controller earns its keep.

    ``controller`` is called as ``controller(ns_queue, ew_queue, serving)`` and
    returns the green duration (seconds) for the phase about to run; it is
    clamped to ``[min_green, max_green]``.
    """
    rng = random.Random(seed)
    ns_q = ew_q = 0.0
    total_delay = 0.0
    served = 0.0
    max_queue = 0.0
    t = 0
    serving = 0          # 0 = NS has green, 1 = EW
    interval = 0

    def rates() -> tuple[float, float]:
        return demand[min(interval, len(demand) - 1)]

    total_seconds = len(demand) * max_green
    while t < total_seconds:
        ns_rate, ew_rate = rates()
        green = controller(ns_q, ew_q, serving)
        green = max(min_green, min(max_green, int(green)))

        for step in range(green + lost_time):
            # Poisson-ish arrivals: a fractional rate as a Bernoulli per second
            ns_q += 1 if rng.random() < ns_rate else 0
            ew_q += 1 if rng.random() < ew_rate else 0
            # The startup lost time at the head of each green discharges nothing
            # - drivers reacting, vehicles accelerating - which is the real cost
            # that makes very short greens inefficient.
            if step >= lost_time:
                if serving == 0:
                    d = min(ns_q, sat_flow)
                    ns_q -= d
                    served += d
                else:
                    d = min(ew_q, sat_flow)
                    ew_q -= d
                    served += d
            total_delay += ns_q + ew_q
            max_queue = max(max_queue, ns_q, ew_q)
            t += 1
        serving ^= 1
        interval += 1

```

```

return IsolatedResult(total_delay, total_delay / max(served, 1.0),
                      max_queue, served)

```

Both simulators report through small result records, so a controller is scored by exactly the fields the experiments read.

Listing 2.3 - the arterial result record

src/cadence/trafficsim.py

```

class ArterialResult:
    total_delay: float      # vehicle-seconds, all approaches
    arterial_delay: float   # vehicle-seconds on the main street only
    cross_delay: float
    arterial_stops: int     # platoon arrivals that met a red
    vehicles_served: float

    def as_dict(self) -> dict:
        return {"total_delay": round(self.total_delay, 1),
                "arterial_delay": round(self.arterial_delay, 1),
                "cross_delay": round(self.cross_delay, 1),
                "arterial_stops": self.arterial_stops,
                "vehicles_served": round(self.vehicles_served, 1)}

```

2.4 The honest negative test

Just as important as the win is a test that guards against overclaiming: on symmetric demand, where there is no asymmetry to exploit, fuzzy must not badly regress against a fixed split. A soft-computing method that quietly hurt the easy case would not be worth deploying.

Listing 2.6 - fuzzy must not regress on symmetric demand

tests/test_trafficsim.py

```

def test_symmetric_demand_no_worse():
    print("-- on symmetric demand fuzzy should not be worse than fixed")
    demand = [(0.3, 0.3)] * 10
    fixed = simulate_isolated(FixedGreenController(25), demand, sat_flow=1.0, seed=3).total_delay
    fuzzy = simulate_isolated(FuzzyGreenController(), demand, sat_flow=1.0, seed=3).total_delay
    # allow a small margin; the point is fuzzy does not badly regress when
    # there is no asymmetry to exploit
    check("fuzzy within 20% of fixed on symmetric demand",
          fuzzy <= fixed * 1.2, f"{fuzzy} vs {fixed}")

```

2.5 Pinning progression in a test

The simulator's central property - that a green wave beats no coordination beats an anti-wave - is asserted directly, so a change that broke progression would fail the suite rather than quietly degrade a report.

Listing 2.5 - asserting the progression ordering

tests/test_trafficsim.py

```
def test_progression_ordering():
    print("-- a green wave beats no coordination beats an anti-wave")
    wave = simulate_arterial([0, 15, 30, 45], **ARTERIAL).total_delay
    zero = simulate_arterial([0, 0, 0, 0], **ARTERIAL).total_delay
    anti = simulate_arterial([0, 45, 30, 15], **ARTERIAL).total_delay
    check("green wave has lowest delay", wave < zero, f"{wave} vs {zero}")
    check("anti-wave has highest delay", anti > zero, f"{anti} vs {zero}")
```

2.6 The isolated result, and the modelling choice

The isolated simulator reports through its own small record - total and average delay, the peak queue, and throughput - so the fuzzy and fixed controllers are compared on identical terms.

Listing 2.4 - the isolated-intersection result record

src/cadence/trafficim.py

```
class IsolatedResult:
    total_delay: float
    avg_delay: float
    max_queue: float
    vehicles_served: float

    def as_dict(self) -> dict:
        return {"total_delay": round(self.total_delay, 1),
                "avg_delay": round(self.avg_delay, 3),
                "max_queue": round(self.max_queue, 1),
                "vehicles_served": round(self.vehicles_served, 1)}
```

Choosing a queue model over a car-following microsimulation is the central modelling decision, and it is a deliberate trade. The queue model is deterministic and fast enough to serve as an objective an optimiser calls thousands of times, and its delay has an exact meaning. It cannot represent shockwaves, acceleration, or lane changes - so the findings are about signal timing and coordination, which it captures faithfully, and not about vehicle dynamics, which it does not attempt. Naming that line is part of reporting the results honestly.

2.7 Little's law, and why delay is an integral

The simulator reports total delay as the sum of the queue length over every time step. That is not an arbitrary proxy for congestion; it is the quantity a driver experiences, and Little's law is why. For any stable queue, the long-run average number waiting equals the arrival rate times the average wait:

$$L = \lambda W$$

Read it the other way. The average wait W is the average queue length L divided by the arrival rate. The total person-seconds of delay over a horizon is then the arrival rate times the horizon times W , which is just the arrival rate times the horizon times L over the arrival rate - the rates cancel, and what remains

is L summed over time. So the area under the queue-length curve *is* total delay, in vehicle-seconds, with nothing assumed about the arrival distribution beyond stability. That is the one number the whole project optimises, and it is why a controller that clears a queue a few seconds sooner on every cycle compounds into a large delay reduction over a horizon of hundreds of steps.

It also explains why coordination matters so much more than raw capacity. A fixed-time plan that serves the same vehicles still builds a taller queue at each stop line, because every vehicle that arrives on red waits the full red. A green wave serves the same flow but keeps L near zero along the arterial, and the integral collapses. The store-and-forward model captures exactly this - the build-up and discharge of L at each approach - which is why it is the right abstraction for a signal-timing study even though it knows nothing about individual cars.

2.8 Saturation flow and startup lost time

A stop line does not discharge at full rate the instant the light turns green. The first few vehicles are still reacting, accelerating from rest, and the flow across the line climbs over the first seconds before it settles at the saturation rate - the steady stream of a moving platoon. Traffic engineering captures this with a startup lost time: a fixed number of seconds at the head of every green that discharge nothing, after which the approach clears at saturation flow until the queue is gone or the green ends.

The simulator models exactly this, and it is a genuine bug fix in this project's history that it does. An earlier version had a discharge helper that always returned true, so every green cleared at full rate from second one - which flatters short greens, because it hands them free capacity they would not have on the street. The corrected inner loop only discharges once the step count passes the lost time, so a green of ten seconds with two seconds of lost time gets eight seconds of real service, not ten. That matters for the fuzzy comparison: the penalty for chopping a phase into many short greens is now real, which is part of why the best fixed split landed at green = 25 rather than something shorter, and why the fuzzy controller's willingness to hold a long queue's green pays off instead of being punished by repeated startup losses.

The number matters more than it looks. Two seconds of lost time on a sixty-second cycle is over three percent of every cycle gone before a wheel turns, and on an arterial of four intersections that tax is paid four times. A model that ignores it would rank timing plans in the wrong order, preferring splits that churn the phase; modelling it is what makes the delay numbers in the reports trustworthy rather than merely internally consistent.

2.9 The region where the model is trustworthy

A store-and-forward queue model is an abstraction, and an abstraction is only useful if you know where it holds. Cadence's model tracks, at each approach, a queue that grows with arrivals and shrinks at saturation flow while the light is green. It is faithful in the regime this project lives in - undersaturated to near-saturated arterials where the question is how to time and coordinate signals - and it is explicitly not faithful outside it.

- ['It captures delay as the integral of the queue, progression as the travel-time offset between stop lines, and the fragmentation of a green wave when offsets are wrong - the three things a signal-timing study must get right.', 'It does not model car-following, acceleration profiles, lane changes or turning movements; a vehicle is a unit of queue, not a body in space, so anything that depends on the shape of individual trajectories is out of scope by construction.', 'It assumes flow does not exceed saturation for long - the queue is allowed to build within a cycle and clear, but sustained oversaturation that spills back past an upstream intersection is a regime the single-queue abstraction stops describing.']

Naming the boundary is not a hedge; it is what lets the findings be read correctly. The 58 percent fuzzy delay reduction and the green-wave optimum are claims about coordination and split timing, which is exactly what the model resolves, so they transfer. A claim about, say, emissions or queue spillback would not transfer, because the model cannot see those things - and the book does not make such a claim. The discipline of stating what an experiment does not establish is the same discipline that lets you trust what it does: the store-and-forward model is the right tool for the signal-timing question, used inside its valid region and nowhere past it.

CHAPTER 3

Fuzzy green time

A Mamdani inference system - fuzzify, fire the rule base, defuzzify - that beats the best-tuned fixed split by lengthening green for a long queue and yielding early for a short one.

3.1 Fuzzify: membership by degree

Each queue length is turned into membership grades in overlapping sets - short, medium, long - so a queue of nine is partly medium and partly long rather than crisply one. The sets are triangles and shoulders, and they overlap on purpose: the overlap is what lets the output vary smoothly as the input crosses a boundary, instead of jumping.

Listing 3.1 - a triangular membership function

src/cadence/fuzzy.py

```
def triangular(x: float, a: float, b: float, c: float) -> float:
    """Membership in a triangular set with feet at a, c and peak at b."""
    if x <= a or x >= c:
        return 0.0
    if x == b:
        return 1.0
    if x < b:
        return (x - a) / (b - a)
    return (c - x) / (c - b)
```

3.2 Infer and defuzzify

A rule base of nine rules maps the two queues - this phase's and the opposing one - to a green length. Each rule's firing strength is the minimum of its antecedent grades (fuzzy AND); the rules are aggregated by maximum per consequent; and the crisp green time is the centroid of the aggregate over singleton outputs.

$$g = \frac{\sum_r w_r g_r}{\sum_r w_r}$$

Listing 3.2 - the Mamdani inference, end to end

src/cadence/fuzzy.py

```

def infer(self, my_queue: float, other_queue: float) -> float:
    mine = self._memberships(my_queue)
    other = self._memberships(other_queue)

    # Rule base. Each rule's firing strength is the min of its antecedents
    # (fuzzy AND), and it votes for a green length.
    rules = [
        (min(mine["short"], other["short"]), "short"),
        (min(mine["short"], other["medium"]), "short"),
        (min(mine["short"], other["long"]), "short"),
        (min(mine["medium"], other["short"]), "medium"),
        (min(mine["medium"], other["medium"]), "medium"),
        (min(mine["medium"], other["long"]), "short"),
        (min(mine["long"], other["short"]), "long"),
        (min(mine["long"], other["medium"]), "long"),
        (min(mine["long"], other["long"]), "medium"),
    ]

    # Aggregate by consequent (max), then defuzzify by centroid over the
    # singleton outputs: sum(strength * green) / sum(strength).
    strength = {"short": 0.0, "medium": 0.0, "long": 0.0}
    for w, label in rules:
        strength[label] = max(strength[label], w)

    num = sum(strength[k] * self.GREEN[k] for k in strength)
    den = sum(strength.values())
    if den == 0:
        return self.GREEN["medium"] # no rule fired; a safe default
    return num / den

```

3.3 The finding, against a fair baseline

The baseline is the best fixed split, found by sweeping the constant green and keeping the lowest-delay one - so the comparison is fair. On demand that is heavy first on the main street and then on the cross street:

Table 3.1 - fuzzy versus the best-tuned fixed split

Controller	Total delay (veh-s)
Best fixed split (green = 25 s)	9,392
Mamdani fuzzy	3,917

A 58% DELAY REDUCTION, OVER THE BEST FIXED TIMING

The fuzzy controller cuts delay by 58% against the best constant split it could be compared with - not an arbitrary one. It does it by holding a long green when its own queue is long and the cross queue short, and yielding early otherwise, which a fixed split cannot do once demand shifts. The full membership sweep is in the report, so the baseline's fairness is checkable.

3.4 Shoulders, the rule base, and the baseline

Besides the triangle, the extreme sets use shoulders - a short queue is fully "short" at zero and stays so up to a knee, and a long queue is fully "long" past its knee. Together the three sets cover the range with overlap everywhere.

Listing 3.3 - the left shoulder

src/cadence/fuzzy.py

```
def shoulder_low(x: float, b: float, c: float) -> float:
    """A left shoulder: full membership up to b, sloping to zero at c."""
    if x <= b:
        return 1.0
    if x >= c:
        return 0.0
    return (c - x) / (c - b)
```

Listing 3.4 - fuzzifying a queue into three grades

src/cadence/fuzzy.py

```
def _memberships(self, q: float) -> dict:
    return {
        "short": shoulder_low(q, 4.0, 10.0),
        "medium": triangular(q, 4.0, 10.0, 16.0),
        "long": shoulder_high(q, 10.0, 18.0),
```

The controller chooses which queue is "mine" from the phase about to run, and the fixed baseline simply returns a constant - the thing the fuzzy system has to beat.

Listing 3.5 - the fixed baseline controller

src/cadence/fuzzy.py

```
class FixedGreenController:
    """The baseline: a constant green regardless of demand."""

    def __init__(self, green: float = 25.0):
        self.green = green

    def __call__(self, ns_queue: float, ew_queue: float, serving: int) -> float:
        return self.green
```

Fuzzy logic control is not a toy: it has run the Sendai city subway's acceleration and braking since 1987 - smoother than a human driver - and sits inside camera autofocus, rice cookers and cement kilns. Its appeal is exactly what this chapter shows: it turns an expert's vague rules ("if my queue is long and theirs is short, hold a long green") directly into a controller, with no model of the plant required.

3.5 Testing the response shape

The controller's qualitative behaviour is pinned too: a longer own queue must lengthen the green, a longer opposing queue must shorten it, and the output must stay in range.

Listing 3.9 - the green responds in the right direction

tests/test_fuzzy.py

```
def test_green_responds_to_queues():
    print("-- green time rises with own queue, falls with the opposing queue")
    f = FuzzyGreenController()
    check("longer own queue -> longer green",
          f.infer(16, 4) > f.infer(8, 4) > f.infer(2, 4),
          f"{f.infer(16,4)}, {f.infer(8,4)}, {f.infer(2,4)}")
    check("longer opposing queue -> shorter green (own fixed)",
          f.infer(10, 16) <= f.infer(10, 2),
          f"{f.infer(10,16)} vs {f.infer(10,2)}")
    check("output stays within the green-time range",
          all(10 <= f.infer(a, b) <= 45 for a in (0, 8, 20) for b in (0, 8, 20)))
```

3.6 The fuzzy claim, as a test

The fuzzy finding is pinned against the same fair baseline the report uses - the best fixed split, found by a sweep - so the test would catch any regression that let a fixed timing match it.

Listing 3.8 - fuzzy must beat the best fixed split

tests/test_trafficsim.py

```
def test_fuzzy_beats_best_fixed():
    print("-- fuzzy control beats the best-tuned fixed split")
    demand = [(0.45, 0.08)] * 6 + [(0.08, 0.45)] * 6
    best_fixed = min(
        simulate_isolated(FixedGreenController(g), demand, sat_flow=1.0, seed=7).total_delay
        for g in range(10, 51, 5)
    )
    fuzzy = simulate_isolated(FuzzyGreenController(), demand, sat_flow=1.0, seed=7).total_delay
    check("fuzzy delay is below the best fixed split", fuzzy < best_fixed,
          f"{fuzzy} vs {best_fixed}")
```

3.7 A worked inference

Take a concrete case: this phase's queue is 12 vehicles, the opposing queue is 4. Fuzzification gives the grades below - the 12 is mostly "medium" with a little "long", the 4 is fully "short".

Table 3.2 - fuzzified grades

Queue	short	medium	long
mine = 12	0.00	0.67	0.25
other = 4	1.00	0.00	0.00

Each rule fires at the minimum of its antecedents. Only three rules fire at all: (medium, short) at 0.67 voting medium; (long, short) at 0.25 voting long; the rest at zero. Aggregating by consequent gives medium = 0.67, long = 0.25, short = 0, and the centroid over the singleton greens (10, 25, 45) is:

$$g = \frac{0.67 \times 25 + 0.25 \times 45}{0.67 + 0.25} \approx 30.5$$

So the controller holds a 30.5-second green - longer than the medium default, because the long queue pulls the average up, but not the full 45, because the queue is only partly "long". That graded response is exactly what the overlap of the sets produces, and what a crisp rule set could not.

3.8 The right shoulder, and dispatching by phase

The high set mirrors the low one, and the controller's call operator simply picks which queue is "mine" from the phase about to run, then defers to the same inference either way.

Listing 3.6 - the right shoulder

src/cadence/fuzzy.py

```
def shoulder_high(x: float, a: float, b: float) -> float:
    """A right shoulder: zero up to a, rising to full at b and staying there."""
    if x <= a:
        return 0.0
    if x >= b:
        return 1.0
    return (x - a) / (b - a)
```

Listing 3.7 - dispatching inference by the serving phase

src/cadence/fuzzy.py

```
def __call__(self, ns_queue: float, ew_queue: float, serving: int) -> float:
    if serving == 0:
        return self.infer(ns_queue, ew_queue)
    return self.infer(ew_queue, ns_queue)
```

Centroid defuzzification is one of several choices - mean-of-maxima and bisector are others - and it is the most common because it uses the whole aggregated output rather than just its peak, so every rule that fired has a proportional say. With singleton consequents it reduces to a firing-strength-weighted average, which keeps the arithmetic transparent while remaining a true Mamdani centroid.

3.9 Fuzzy is not probability

A membership of 0.7 in the set "long queue" is not a 70 percent chance the queue is long. The queue length is known exactly; the 0.7 is the *degree* to which that exact, certain length belongs to a vague linguistic category. The distinction is not pedantic - it changes the arithmetic. Probabilities of mutually exclusive events sum to one and combine by multiplication under independence. Membership degrees obey no such constraint: a queue of twelve vehicles can be 0.6 "medium" and 0.5 "long" at once, and those do not sum to one because the categories overlap by design.

Combination follows suit. Fuzzy logic uses min for AND and max for OR - the operations Cadence's nine rules use - precisely because they are the pointwise-consistent generalisation of Boolean AND and OR to the interval [0, 1], not because they approximate any joint distribution. The result is a controller that degrades gracefully: as the queue grows from medium to long, the green time it commands rises smoothly, because two overlapping rules fire with shifting weights and the centroid slides between

their singletons. A probabilistic controller would instead give you the expected green under some assumed distribution of queue length, which is a different and less useful object when the length is measured, not guessed.

This is the honest reason the problem suits fuzzy control. The uncertainty here is linguistic, not aleatory: "heavy demand" is a vague word applied to a precise measurement, and fuzzy membership is the tool built for vague words. Where the uncertainty were genuinely about chance - will a detector fire this second - a probabilistic model would be right, and the book would not reach for Mamdani inference.

3.10 Why centroid, and not mean-of-maxima

Defuzzification turns the aggregated fuzzy output back into a single green time, and the choice of method is not cosmetic. Cadence uses the centroid - the balance point of the output, here a weighted average of the rule singletons by their firing strengths. The alternative most often taught, mean-of-maxima, takes the average of the outputs where membership is highest and throws the rest away.

The centroid is the right choice for a controller that must move smoothly. Because it is a weighted average over every rule that fires, a small change in the queue - nudging one rule's strength up and another's down - slides the green time continuously. Mean-of-maxima, by contrast, can jump: as the dominant rule changes, the output can leap from one singleton to another with no intermediate values, and a signal controller that jumps its green time between discrete levels as the queue drifts is both uncomfortable and harder to reason about. The worked inference earlier in this chapter - two rules firing at 0.67 and 0.25, blending to a green near 30 seconds - is exactly the smooth interpolation the centroid provides and the mean-of-maxima would have collapsed to a flat 45.

There is a cost: the centroid pulls toward the middle, so a fuzzy system using it rarely commands the extreme greens even when one rule fires alone, because the membership tails of its neighbours still tug the balance point inward. For traffic control that conservatism is a feature - it damps overreaction to a transient spike - but it is the kind of trade-off a designer should make on purpose, which is why the book names it rather than presenting the centroid as the only option.

CHAPTER 4

Coordinating the network: GA versus grid search

The headline comparison. A genetic algorithm reaches the exhaustive optimum in a fraction of the simulations - and the honest reading is that this matters more the larger the network.

4.1 The objective, and the exhaustive baseline

Only relative offsets matter, so the first intersection is pinned at zero and grid search enumerates every offset vector for the rest. On a small arterial that is a complete search - it finds the true optimum, at a cost of exactly one simulation per grid point, which is the yardstick a heuristic has to earn its place against.

Listing 4.1 - exhaustive grid search

src/cadence/gridsearch.py

```
def search(objective, n: int, cycle: int, step: int) -> dict:
    """Evaluate `objective(offsets) -> delay` at every grid point and keep the
    best. Returns the optimum, its delay, and the number of evaluations - which
    for grid search is simply the grid size."""
    best_offsets = None
    best_delay = float("inf")
    evaluations = 0
    for offsets in offset_grid(n, cycle, step):
        delay = objective(offsets)
        evaluations += 1
        if delay < best_delay:
            best_delay = delay
            best_offsets = offsets
    return {"method": "grid_search", "best_offsets": best_offsets,
            "best_delay": round(best_delay, 1), "evaluations": evaluations}
```

4.2 The genetic algorithm

A chromosome is an offset vector; fitness is the negative of total delay. Selection is by tournament, crossover is uniform, and mutation nudges an offset to another grid value. Crucially the algorithm caches every vector it has simulated, so the evaluations it reports are distinct simulations - the fair quantity to set beside grid search's exhaustive count.

Listing 4.2 - the generational loop, with elitism

src/cadence/genetic.py

```

pop = [random_chrom() for _ in range(population)]
best = min(pop, key=evaluate)
best_delay = evaluate(best)
best_gen = 0

for gen in range(1, generations + 1):
    new_pop = [best]           # elitism: carry the best forward
    while len(new_pop) < population:
        child = mutate(crossover(select(pop), select(pop)))
        new_pop.append(child)
    pop = new_pop
    gen_best = min(pop, key=evaluate)
    if evaluate(gen_best) < best_delay:
        best = gen_best
        best_delay = evaluate(gen_best)
        best_gen = gen

```

4.3 The result, stated honestly

Table 4.1 - fixed, grid search, and the GA on the arterial

Method	Best delay	Simulations
Fixed (zero offsets)	44,290	—
Grid search (exhaustive)	24,480	1,728
Genetic algorithm	24,480	177

The GA reaches grid search's *exact* optimum - offsets [0, 15, 30, 45], a green wave matching the travel time - using 90% fewer simulations. It ties on quality and wins on time.

THE WIN IS SCALING, AND THE BOOK SAYS SO

On a space this small, grid search runs in seconds, so the GA's edge is real but modest. It becomes decisive with size: the grid is (cycle/step) to the power (n-1) points, so at 6 intersections an exhaustive search would be 248,832 simulations, while the GA found a strong solution in 825. The honest summary is that the GA did not beat grid search on quality - it matched it - and its value is that it keeps that quality affordable as the exhaustive search explodes.

4.4 The GA operators, and the grid

The genetic operators are small and standard. A chromosome is generated with the first offset pinned; mutation nudges the others to random grid values; uniform crossover mixes two parents gene by gene; and tournament selection picks the fitter of a small random sample.

Listing 4.3 - mutation, crossover, and tournament selection

src/cadence/genetic.py

```
def mutate(chrom: list[int]) -> list[int]:
    child = list(chrom)
    for i in range(1, n):          # never mutate the pinned first offset
        if rng.random() < mutation_rate:
            child[i] = rng.choice(values)
    return child

def crossover(a: list[int], b: list[int]) -> list[int]:
    return [0, *[a[i] if rng.random() < 0.5 else b[i] for i in range(1, n)]]

def select(pop: list[list[int]]) -> list[int]:
    contenders = rng.sample(pop, tournament)
    return min(contenders, key=evaluate)
```

The grid the exhaustive search enumerates is every offset combination on a fixed step, with the first intersection pinned - the search space both methods draw from.

Listing 4.4 - the offset grid

src/cadence/gridsearch.py

```
def offset_grid(n: int, cycle: int, step: int) -> list[list[int]]:
    """All offset vectors with intersection 0 pinned at 0 and the others on a
    grid of `step`-second increments across the cycle."""
    values = list(range(0, cycle, step))
    return [[0, *rest] for rest in itertools.product(values, repeat=n - 1)]
```

Adaptive network coordination is real infrastructure: Sydney's SCATS and the UK's SCOOT have adjusted signal timings and offsets from live detector data since the 1980s, across thousands of intersections. They do not run a genetic algorithm, but they solve the same problem this chapter poses - choose offsets to move platoons through - and they face the same combinatorial explosion that makes an exhaustive search hopeless at city scale and a heuristic necessary.

4.5 The GA's report, and its cache test

The algorithm returns not just the answer but its cost and how quickly it converged, so the comparison is fully accountable.

Listing 4.7 - what the GA reports

src/cadence/genetic.py

```
return {"method": "genetic_algorithm", "best_offsets": best,
        "best_delay": round(best_delay, 1),
        "evaluations": len(cache),
        "converged_generation": best_gen,
        "population": population, "generations": generations}
```

And a test confirms the cache does what the fairness of the comparison depends on: distinct simulations equal the reported evaluation count.

Listing 4.8 - the evaluation count is distinct simulations

tests/test_optimisers.py

```
def test_ga_caches():
    print("-- the GA does not re-simulate an offset it has already seen")
    calls = {"n": 0}

    def counting_obj(offsets):
        calls["n"] += 1
        return objective(offsets)

    ga = genetic.optimise(counting_obj, 4, 60, 5, seed=2)
    check("distinct simulations equal the reported evaluation count",
          calls["n"] == ga["evaluations"], f"{calls['n']} vs {ga['evaluations']}")
```

4.6 The headline comparison, as a test

The claim the whole chapter rests on - the GA reaches grid search's optimum in far fewer evaluations - is a single assertion, so it cannot silently rot.

Listing 4.6 - the GA matches the optimum for fewer simulations

tests/test_optimisers.py

```
def test_ga_matches_grid_cheaper():
    print("-- the GA reaches the grid optimum in fewer evaluations")
    grid = gridsearch.search(objective, 4, 60, 5)
    ga = genetic.optimise(objective, 4, 60, 5, seed=1)
    check("GA reaches the grid optimum",
          ga["best_delay"] <= grid["best_delay"] * 1.001,
          f"GA {ga['best_delay']} vs grid {grid['best_delay']}")
    check("GA used far fewer evaluations than grid",
          ga["evaluations"] < grid["evaluations"] / 2,
          f"{ga['evaluations']} vs {grid['evaluations']}")
```

4.7 What a green wave really is

The optimum the search finds - offsets stepping up by the travel time - is a green wave: a band of green that moves down the arterial at the speed of the platoon, so a vehicle catching the first green rides the rest without stopping. Traffic engineers measure it as bandwidth, the width of that through-band, and maximising bandwidth is the classic arterial-coordination objective. Minimising total delay, as here, is a close relative that also credits the cross street, which is why the optimum lands on the travel-time-matched offsets a bandwidth method would also choose. The point of interest is not that the answer is a green wave - that is known - but that the genetic algorithm rediscovers it from delay alone, with no green-wave heuristic built in.

4.8 Fitness caching, and why not annealing

The single most important line in the GA for a fair comparison is the cache: an offset vector is simulated once and remembered, so a re-visited chromosome costs nothing and the reported evaluation count is distinct simulations only.

Listing 4.5 - the fitness cache and random chromosome

src/cadence/genetic.py

```
def evaluate(chrom: list[int]) -> float:
    key = tuple(chrom)
    if key not in cache:
        cache[key] = objective(chrom)
    return cache[key]

def random_chrom() -> list[int]:
    return [0, *[rng.choice(values) for _ in range(n - 1)]]
```

A genetic algorithm is not the only heuristic that would work here - simulated annealing or a simple local search might match it - and the appendix names that as the honest next comparison. The GA was chosen because offsets combine naturally under crossover (two good partial coordinations can merge into a better whole), but the project does not claim it is the best heuristic, only that it beats the exhaustive baseline on time and scales.

4.9 Why tournament selection and uniform crossover

A genetic algorithm is a set of choices, and each one in Cadence was made for a reason the results can defend. Selection is by tournament, not roulette. Roulette - sampling parents with probability proportional to fitness - has two failures this problem would hit: it is sensitive to the scale of the fitness numbers, and delay values of 24,000 against 44,000 are close in ratio, so roulette would barely prefer the better offsets; and one early lucky individual can dominate the roulette wheel and collapse diversity. A tournament - pick k at random, keep the best - depends only on rank, so it is invariant to how the delays are scaled and gives a steady, tunable selection pressure through k .

Crossover is uniform, not single-point. Single-point crossover assumes that genes near each other on the chromosome belong together, so cutting between them is cheap. For an offset vector there is no such ordering - the offset at intersection one is no more linked to intersection two's than to intersection four's - so any fixed cut point would impose a structure the problem does not have. Uniform crossover, taking each gene independently from either parent, respects that the genes are exchangeable. Mutation then perturbs a single offset to a nearby grid value, which is the local search that lets the population refine a good green wave rather than only recombine existing ones.

- ['Elitism carries the best individual forward unchanged, so the best delay can never increase from one generation to the next - the convergence curve in the report is monotone by construction.', 'The fitness cache means a re-created offset vector is never re-simulated, so the reported evaluation count is the number of **distinct** simulations, which is the only honest way to compare cost against the exhaustive grid.', 'Selection pressure, population size and mutation rate trade exploration against exploitation; the values used converge well before the generation budget, which is why the run reports a convergence generation rather than exhausting it.']

4.10 What the exhaustive guarantee is worth

Grid search is the baseline precisely because it cannot be beaten on quality within the discretised space. It evaluates every offset combination on the grid and returns the best, so whatever it finds is the true optimum to the resolution of the grid - there is no better answer to be had at that step size, by definition. That is a strong and honest yardstick, and it is why the project insists the genetic algorithm justify itself against it rather than against a weak heuristic that would make any method look good.

But the guarantee has two edges. It holds only on the discretised space: with a five-second step the grid can miss a better offset sitting at twelve seconds, so both grid and GA are optimal-on-the-grid, not optimal-in-reality. Refining the grid to one-second steps would recover those offsets - and there the cost becomes the whole point. The grid is (cycle / step) to the power (n - 1) points, so halving the step does not double the work, it raises it by a factor of two to the (n - 1): at four intersections a one-second grid is already sixty cubed, over two hundred thousand simulations, and at six intersections it is astronomical.

This is where the GA's scaling argument becomes concrete rather than rhetorical. The heuristic's cost grows roughly linearly in the number of intersections - more genes, a few more generations - while the grid's grows as a power of them. On the four-intersection arterial at five-second resolution the grid is 1,728 simulations and perfectly affordable, which is exactly why the book refuses to claim a win there: the GA merely ties. The win is the slope. Every intersection added, or every refinement of the grid, multiplies the exhaustive cost and barely moves the heuristic's, and that is the only thing the GA is allowed to claim here - earned on time-to-solution, not on quality.

4.11 When the heuristic would not help

An honest account of a method includes the cases it does not fit, and the genetic algorithm has them. On the four-intersection arterial the book is explicit that the GA only ties grid search, because the space is small enough to enumerate; there, reaching for a heuristic is over-engineering, and the right answer is the exhaustive one. The GA earns its keep only when the space is too large to enumerate, which is a statement about the network, not the algorithm.

There are harder limits too. A genetic algorithm has no guarantee of optimality at all - it can converge to a local optimum and stop, and on a different objective it might return a worse answer than grid search while costing more than a simple hill-climb. It carries its own budget of choices - population size, selection pressure, mutation rate - each of which can be set badly, and a mis-tuned GA is easy to beat. The reason it works cleanly here is that the offset-coordination objective is well-behaved: relatively smooth, with a clear green-wave optimum that a population can find and elitism can hold. Hand it a rugged, deceptive objective and the same code would struggle.

The general lesson is the no-free-lunch one: averaged over all possible objectives, no search method beats any other, so a heuristic's value is always a claim about a **particular** class of problems. Cadence's claim is narrow and defensible - on signal-offset coordination, where the space grows as a power of the network size and the landscape is benign, the GA scales where the exhaustive baseline cannot. That is the whole of the claim, and the book declines to inflate it into a general superiority the measurements would not support.

CHAPTER 5

A Hopfield network for noisy detectors

An associative memory that stores traffic-state patterns as attractors and relaxes a corrupted detector reading back to the nearest one - up to a capacity limit it measures.

5.1 Hebbian storage and asynchronous recall

The detectors across the network report a pattern - which approaches are congested, which are clear. A Hopfield network stores the typical patterns as fixed points of a recurrent dynamics, using the Hebbian outer-product rule, and recalls one by relaxing a noisy input to the nearest fixed point.

$$W_{ij} = \frac{1}{N} \sum_p x_i^p x_j^p, \quad W_{ii} = 0$$

Listing 5.1 - Hebbian storage

src/cadence/hopfield.py

```
def train(self, patterns: np.ndarray) -> None:
    """Hebbian storage:  $W = (1/N) \sum_p x_p x_p^T$ , with a zero diagonal so a
    neuron does not reinforce itself. Patterns are bipolar (-1/+1) rows."""
    self.W = np.zeros((self.size, self.size))
    for p in patterns:
        self.W += np.outer(p, p)
    self.W /= self.size
    np.fill_diagonal(self.W, 0.0)
```

Listing 5.2 - asynchronous relaxation to a fixed point

src/cadence/hopfield.py

```
def recall(self, x: np.ndarray, max_steps: int = 20,
           rng: np.random.Generator | None = None) -> np.ndarray:
    """Relax a state to a fixed point by asynchronous updates: repeatedly
    set a random neuron to the sign of its weighted input, until a full
    sweep changes nothing. Asynchronous (one neuron at a time) is what
    guarantees convergence for a symmetric zero-diagonal weight matrix."""
    rng = rng or np.random.default_rng(0)
    s = x.copy().astype(float)
    for _ in range(max_steps):
        changed = False
        for i in rng.permutation(self.size):
            net = self.W[i] @ s
            new = 1.0 if net >= 0 else -1.0
            if new != s[i]:
                s[i] = new
                changed = True
        if not changed:
            break
    return s
```

5.2 Recovery, and the capacity cliff

Two measurements, both over 200 trials per point. The recovery curve shows how much corruption a stored pattern survives; the capacity curve shows how many patterns a network can hold before the attractors interfere.

Table 5.1 - recovery versus corruption (N = 64)

Bits flipped	Fraction	Exact recovery
0	0.00	1.00
5	0.08	1.00
10	0.16	1.00
15	0.23	1.00
20	0.31	0.94
25	0.39	0.73
30	0.47	0.12

Table 5.2 - recovery versus load

Patterns	Load ratio	Recovery
1	0.016	1.00
4	0.062	1.00
7	0.109	1.00
10	0.156	0.77
13	0.203	0.78
16	0.250	0.45
19	0.297	0.07

THE 0.138 N CAPACITY, MEASURED

Recovery is exact up to roughly a quarter of the bits flipped, then falls away. And the network recovers reliably only until the stored patterns exceed a load ratio of about 0.156, close to the classic theoretical bound of $0.138 N$ - past which the attractors overlap and recall collapses. The cliff is measured here, not quoted.

5.3 Corruption, and measuring recovery

Sensor noise is modelled as flipping bits of a bipolar pattern; the recovery experiment stores a few patterns, corrupts one by a growing number of flips, relaxes it, and counts exact recoveries over many trials.

Listing 5.3 - flipping bits to model sensor noise

src/cadence/hopfield.py

```
def corrupt(pattern: np.ndarray, flips: int,
            rng: np.random.Generator) -> np.ndarray:
    """Flip `flips` random bits of a bipolar pattern - modelling sensor noise."""
    out = pattern.copy()
    idx = rng.choice(len(pattern), size=flips, replace=False)
    out[idx] *= -1
    return out
```

Listing 5.4 - the recovery-versus-corruption experiment

src/cadence/hopfield.py

```
def recovery_curve(size: int, n_patterns: int, seed: int,
                  trials: int = 200) -> list[dict]:
    """Store `n_patterns` random patterns, then measure exact-recovery rate as
    the number of flipped bits grows - the associative-memory recovery curve."""
    rng = np.random.default_rng(seed)
    patterns = rng.choice([-1, 1], size=(n_patterns, size))
    net = HopfieldNetwork(size)
    net.train(patterns)

    out = []
    for flips in range(0, size // 2 + 1, max(1, size // 12)):
        recovered = 0
        for _ in range(trials):
            p = patterns[rng.integers(n_patterns)]
            noisy = corrupt(p, flips, rng)
            settled = net.recall(noisy, rng=rng)
            if np.array_equal(settled, p):
                recovered += 1
        out.append({"flipped_bits": int(flips),
                  "fraction_of_bits": round(flips / size, 3),
                  "exact_recovery_rate": round(recovered / trials, 3)})
    return out
```

The capacity experiment instead holds the corruption fixed and grows the number of stored patterns, tracing the cliff where the attractors begin to interfere.

Listing 5.5 - the capacity experiment

src/cadence/hopfield.py

```
def capacity_curve(size: int, seed: int, trials: int = 200) -> list[dict]:
    """Store an increasing number of patterns and measure how reliably a
    lightly-corrupted pattern is recovered - the capacity cliff around
    0.138 N."""
    rng = np.random.default_rng(seed)
    flips = max(1, size // 20) # a light, fixed corruption
    out = []
    for m in range(1, int(0.30 * size) + 1, max(1, size // 20)):
        patterns = rng.choice([-1, 1], size=(m, size))
        net = HopfieldNetwork(size)
        net.train(patterns)
        recovered = 0
        for _ in range(trials):
            p = patterns[rng.integers(m)]
            settled = net.recall(corrupt(p, flips, rng), rng=rng)
            if np.array_equal(settled, p):
                recovered += 1
        out.append({"stored_patterns": m,
                    "load_ratio": round(m / size, 3),
                    "recovery_rate": round(recovered / trials, 3)})
    return out
```

5.4 Testing recall directly

Recall is pinned at both ends: an exact stored pattern is a fixed point, and a lightly corrupted one comes back the great majority of the time.

Listing 5.8 - stored patterns are fixed points; noise is recovered

tests/test_softcomputing.py

```
def test_hopfield_recall():
    print("-- Hopfield recovers stored and lightly-corrupted patterns")
    rng = np.random.default_rng(0)
    patterns = rng.choice([-1, 1], size=(3, 64))
    net = HopfieldNetwork(64)
    net.train(patterns)
    # an exact pattern is a fixed point
    exact = all(np.array_equal(net.recall(p, rng=rng), p) for p in patterns)
    check("stored patterns are fixed points", exact)
    # a lightly corrupted pattern relaxes back
    p = patterns[0]
    recovered = sum(np.array_equal(net.recall(corrupt(p, 5, rng), rng=rng), p)
                    for _ in range(50))
    check("light corruption is recovered most of the time", recovered >= 45,
          f"{recovered}/50")
```

5.5 The capacity cliff, as a test

Both ends of the capacity curve are pinned: near-perfect recall at low load, collapse past it. A silent change to the storage rule would move one of them and fail here.

Listing 5.7 - asserting the capacity collapse

tests/test_softcomputing.py

```
def test_hopfield_capacity():
    print("-- recovery degrades as the network is overloaded")
    cap = capacity_curve(64, seed=1, trials=100)
    low = cap[0]["recovery_rate"]
    high = cap[-1]["recovery_rate"]
    check("recovery is near-perfect at low load", low >= 0.95, str(low))
    check("recovery collapses past capacity", high < 0.5, str(high))
```

5.6 The energy that guarantees convergence

A Hopfield network with symmetric weights and a zero diagonal has an energy function that every asynchronous update can only decrease or leave unchanged. Since the energy is bounded below, the dynamics must reach a local minimum - a stable pattern - which is why recall always terminates rather than oscillating.

$$E = -\frac{1}{2} \sum_i \sum_j W_{ij} S_i S_j$$

Listing 5.6 - the network's state

src/cadence/hopfield.py

```
def __init__(self, size: int):
    self.size = size
    self.W = np.zeros((size, size))
```

The stored patterns are engineered to be exactly those minima. The capacity limit is what happens when too many minima are packed into the same energy landscape: they merge and shift, and the one you wanted stops being a minimum at all.

5.7 A worked recall, by hand

The mechanism is clearest at $N = 4$. Store two patterns, $p1 = (+1, +1, -1, -1)$ and $p2 = (+1, -1, +1, -1)$. The Hebbian rule sums their outer products and zeroes the diagonal, giving:

Table 5.3 - the weight matrix for two 4-bit patterns

	n0	n1	n2	n3
n0	0	0	0	-0.5
n1	0	0	-0.5	0
n2	0	-0.5	0	0
n3	-0.5	0	0	0

Now corrupt p1's first bit to get (-1, +1, -1, -1) and update neuron 0. Its net input is the weighted sum of the others: only $W[0][3] = -0.5$ is non-zero, times $s_3 = -1$, giving $+0.5$. That is at least zero, so neuron 0 flips to +1 - back to p1. The single corrupted bit was pulled into line by the one it was correlated with across the stored patterns, which is associative recall in one step. Scale that to sixty-four neurons and a handful of patterns and it is the detector-recovery result of Table 5.1.

5.8 Spurious states, and the cost of overloading

A Hopfield network does not only store the patterns you train it on. The Hebbian weight matrix also creates *spurious* attractors - stable states that are none of the stored patterns. The simplest are the negations: if p is a fixed point, so is -p, because the energy function is even in the state. More troubling are the mixture states, sign combinations of three or more stored patterns, which appear as the network fills up and sit between the genuine memories.

This is the mechanism behind the capacity cliff the report measures. Below about 0.138 N stored patterns the genuine attractors have wide, clean basins and a corrupted reading relaxes into the right one. As the load climbs past that, spurious mixtures multiply and crowd the state space; a noisy pattern now falls into a mixture instead of its parent, and recall accuracy falls off a cliff rather than degrading gently. Cadence's measured cliff at load 0.156 on a 64-neuron network is close to the theoretical 0.138 - the small excess is exactly what a finite network and a finite trial count produce, and the book reports the measured value rather than rounding it to the textbook one.

The practical lesson for detector recovery is a design constraint, not a flaw: size the network generously relative to the number of distinct traffic-state patterns you need it to hold. A network asked to remember too many states does not fail loudly - it quietly returns a plausible blend that was never a real reading, which for a recovery system is the worst kind of error. Knowing where the cliff is tells you how many patterns you may safely store, and the capacity curve is how you find it for any N.

5.9 Asynchronous updating, and why it settles

Cadence relaxes a corrupted pattern asynchronously: one neuron at a time, in random order, each reading the current state of all the others before it flips. This is not an implementation convenience - it is what guarantees the network settles at all. With a symmetric, zero-diagonal weight matrix and asynchronous updates, every flip either lowers the energy or leaves it unchanged, and because the energy is bounded below the network must reach a fixed point in finite time.

$$E = -\frac{1}{2} \sum_i \sum_j W_{ij} S_i S_j$$

Synchronous updating - flipping every neuron at once from the old state - loses that guarantee. It can fall into a two-cycle, oscillating forever between two states that each map to the other, never landing. The asynchronous rule avoids this because by the time neuron five decides, it has already seen the new value of neuron three; the updates are serialised, so no two can disagree about the state they are

responding to. The random order matters too: a fixed sweep order can bias which attractor a boundary case falls into, so the recovery curve is measured over many trials with the order reshuffled, which is why the reported numbers are rates over hundreds of runs rather than single outcomes.

The energy picture is the intuition behind the whole associative memory. Each stored pattern is a valley; training carves those valleys into the energy surface via the Hebbian rule; recall is a ball rolling downhill from wherever the corrupted reading drops it. Light corruption drops the ball inside the right valley and it rolls home; heavy corruption, or too many stored patterns crowding the surface with spurious dips, drops it into the wrong basin. Convergence is guaranteed; convergence to the *right* pattern is what the recovery and capacity curves measure.

CHAPTER 6

The perceptron, ADALINE, and XOR

Two single-layer learners from scratch, and the boundary neither can cross - the honest limit that explains why deeper networks exist.

6.1 Two learning rules

Both classify a traffic state from two features. The perceptron learns from the sign of its output with the perceptron rule and converges in finite time if the classes are linearly separable. ADALINE learns from the continuous output with the delta (LMS) rule, descending mean-squared error, so it settles smoothly - but it is still a single linear unit.

$$\Delta w = \eta (y - \hat{y}) x$$

Listing 6.1 - the perceptron rule

src/cadence/neurons.py

```
def fit(self, X: np.ndarray, y: np.ndarray, epochs: int = 100) -> dict:
    """The perceptron rule: on a misclassified point, nudge the weights
    toward it. Track the epoch at which errors first hit zero - which
    happens for separable data and never for the rest."""
    converged_epoch = None
    history = []
    for epoch in range(epochs):
        errors = 0
        for xi, target in zip(X, y):
            pred = 1 if xi @ self.w + self.b >= 0 else 0
            update = self.lr * (target - pred)
            self.w += update * xi
            self.b += update
            errors += int(update != 0.0)
        history.append(errors)
        if errors == 0 and converged_epoch is None:
            converged_epoch = epoch
            break
    acc = float(np.mean(self.predict(X) == y))
    return {"converged_epoch": converged_epoch, "final_accuracy": round(acc, 3),
            "error_history": history}
```

6.2 Convergence, and the wall

On linearly separable data the perceptron converges in 4 epochs to perfect accuracy, and ADALINE reaches the same by minimising its error. On XOR, neither can:

Table 6.1 - both learners, separable versus XOR

Problem	Perceptron accuracy	ADALINE accuracy
Linearly separable	1.00	0.99
XOR	0.25	0.25

THE 1969 RESULT, REPRODUCED

No straight line separates XOR, and a single linear unit can draw only a straight line - so neither the perceptron nor ADALINE exceeds chance on it, and the perceptron never converges. This is the limitation that stalled neural networks for a decade and the honest reason multi-layer networks had to be invented. It is shown here by running it, not by citing it.

6.3 ADALINE, and the datasets

ADALINE differs from the perceptron in learning from the continuous activation rather than the thresholded output, and in descending a smooth squared-error cost - so it settles rather than cycling, though it draws the same kind of straight boundary.

Listing 6.2 - the delta (LMS) rule

src/cadence/neurons.py

```
def fit(self, X: np.ndarray, y: np.ndarray, epochs: int = 200) -> dict:
    """The delta rule: descend the mean-squared error of the *continuous*
    output. Because it minimises a smooth cost, it settles rather than
    oscillating - but it still draws only a straight boundary."""
    mse_history = []
    for _ in range(epochs):
        output = self.activation(X)
        error = y - output
        self.w += self.lr * (X.T @ error) / len(X)
        self.b += self.lr * error.mean()
        mse_history.append(float((error ** 2).mean()))
    acc = float(np.mean(self.predict(X) == y))
    return {"final_mse": round(mse_history[-1], 4),
            "final_accuracy": round(acc, 3),
            "mse_history": [round(m, 4) for m in mse_history[:max(1, epochs // 10)]]}
```

The two datasets make the point: a separable traffic-state set both can learn, and XOR, which neither can.

Listing 6.3 - a separable congested/free dataset

src/cadence/neurons.py

```
def linearly_separable(seed: int, n: int = 200) -> tuple[np.ndarray, np.ndarray]:
    """A separable traffic-state dataset: two features (say normalised queue and
    occupancy), class 1 = congested when their sum is high, with a clear gap so
    the classes do not touch."""
    rng = np.random.default_rng(seed)
    free = rng.normal([0.3, 0.3], 0.1, (n // 2, 2))
    congested = rng.normal([0.7, 0.7], 0.1, (n // 2, 2))
    X = np.vstack([free, congested])
    y = np.array([0] * (n // 2) + [1] * (n // 2))
    return X, y
```

Listing 6.4 - the non-separable XOR set

src/cadence/neurons.py

```
def xor_problem() -> tuple[np.ndarray, np.ndarray]:
    """The canonical non-separable set: XOR. No straight line splits it, so no
    single linear unit can classify it correctly."""
    X = np.array([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
    y = np.array([0, 1, 1, 0])
    return X, y
```

6.4 Testing convergence on separable data

Both learners are pinned on the separable case - the perceptron must converge to perfect accuracy, and ADALINE's error must fall - so the XOR failure is clearly a property of the problem, not a broken implementation.

Listing 6.8 - the perceptron converges when separable

tests/test_softcomputing.py

```
def test_perceptron_separable():
    print("-- perceptron converges on linearly separable data")
    X, y = linearly_separable(1)
    r = Perceptron(2, seed=1).fit(X, y)
    check("perceptron converges", r["converged_epoch"] is not None)
    check("perceptron is perfectly accurate when separable",
          r["final_accuracy"] == 1.0, str(r["final_accuracy"]))
```

Listing 6.9 - ADALINE reaches full accuracy and its error falls

tests/test_softcomputing.py

```
def test_adaline_separable():
    print("-- ADALINE minimises MSE and classifies separable data")
    X, y = linearly_separable(1)
    r = Adaline(2, seed=1).fit(X, y)
    check("ADALINE reaches full accuracy when separable",
          r["final_accuracy"] == 1.0, str(r["final_accuracy"]))
    check("ADALINE's MSE decreases", r["mse_history"][0] > r["mse_history"][-1])
```

6.5 The XOR wall, as a test

The limitation is asserted, not just observed: the perceptron must fail to converge on XOR, and neither unit may reach perfect accuracy on it.

Listing 6.7 - pinning the XOR limitation

tests/test_softcomputing.py

```
def test_xor_wall():
    print("-- neither single-layer unit can solve XOR")
    X, y = xor_problem()
    p = Perceptron(2, seed=1).fit(X, y, epochs=200)
    a = Adaline(2, seed=1).fit(X, y, epochs=500)
    check("perceptron never converges on XOR", p["converged_epoch"] is None)
    check("perceptron is below perfect on XOR", p["final_accuracy"] < 1.0,
          str(p["final_accuracy"]))
    check("ADALINE is below perfect on XOR", a["final_accuracy"] < 1.0,
          str(a["final_accuracy"]))
```

6.6 Why the delta rule descends smoothly

The perceptron rule changes weights only on a misclassification, by a fixed step, so it lurches and can circle the boundary. ADALINE's delta rule instead does gradient descent on a smooth cost - the mean-squared error of the continuous activation - so its steps shrink as it approaches the minimum and it settles rather than lurching.

$$J(w) = \frac{1}{2} \sum_i (y_i - w \cdot x_i)^2$$

The gradient of that cost is the sum of error times input, which is exactly the update the code applies; descending it is guaranteed to reach the unique minimum of a convex quadratic. The catch is that the minimum is the best *linear* fit, and when the classes are not linearly separable - XOR - the best linear fit still misclassifies, however perfectly the descent converges. A better optimiser cannot rescue the wrong hypothesis class, which is the lesson the next section makes concrete.

6.7 The two units, and the winter they caused

The perceptron is weights, a bias, and a threshold; ADALINE replaces the threshold-during-training with the raw linear activation.

Listing 6.5 - the perceptron

src/cadence/neurons.py

```
class Perceptron:
    def __init__(self, n_features: int, lr: float = 0.1, seed: int = 0):
        rng = np.random.default_rng(seed)
        self.w = rng.normal(0, 0.01, n_features)
        self.b = 0.0
        self.lr = lr

    def predict(self, X: np.ndarray) -> np.ndarray:
        return np.where(X @ self.w + self.b >= 0, 1, 0)
```

Listing 6.6 - ADALINE's linear activation and its threshold

src/cadence/neurons.py

```
def activation(self, X: np.ndarray) -> np.ndarray:
    return X @ self.w + self.b      # linear, no threshold during training

def predict(self, X: np.ndarray) -> np.ndarray:
    return np.where(self.activation(X) >= 0.5, 1, 0)
```

The XOR result is not a curiosity; it is history. Minsky and Papert's 1969 book *Perceptrons* proved that a single-layer perceptron cannot compute XOR, and the demonstration - reproduced in this chapter by running it - was widely read as a verdict on neural networks in general. Funding collapsed and the first "AI winter" followed; the field only recovered when the backpropagation of the 1980s made multi-layer networks trainable, and a hidden layer is exactly what lets a network bend the boundary XOR needs. The wall this chapter hits is the one that shaped a decade.

6.8 The bias, and what a hidden layer buys

Neither the perceptron nor ADALINE can solve XOR, and the reason is geometric and exact. A single linear unit computes a weighted sum of its inputs plus a bias and thresholds it, which carves the input plane with one straight line. The bias term is what lets that line sit away from the origin - without it every decision boundary would pass through (0, 0), and even some separable problems would be unsolvable. With it, the unit can place its line anywhere, but it is still one line.

$$\hat{y} = \text{sign}(w_1x_1 + w_2x_2 + b)$$

XOR's four points - (0,0) and (1,1) in one class, (0,1) and (1,0) in the other - cannot be split by any single straight line; the two classes sit on opposite diagonals. No choice of the three numbers above separates them, which is why the perceptron never converges and ADALINE's accuracy sticks at chance however long it trains. The experiment reproduces this rather than asserting it, and that is the point: the wall is a theorem, and a from-scratch unit hits it at exactly the predicted place.

A hidden layer dissolves the wall. Two hidden units can each draw one line - one firing for "at least one input on", the other for "both on" - and a third unit combining them computes "at least one but not both", which is XOR. The cost is that the simple perceptron and delta rules no longer suffice to train the hidden weights; you need to propagate the error back through the layer, and that is backpropagation, the algorithm the 1969 result delayed and the 1986 revival delivered. Cadence stops at the single layer on purpose - the book's job is to make the wall concrete, so that the reason the field built over it is felt rather than taken on faith.

6.9 The perceptron convergence theorem

The perceptron solving the separable problem in four epochs is not luck; it is a theorem. Rosenblatt's convergence theorem says that if the two classes are linearly separable, the perceptron learning rule will find a separating hyperplane in a finite number of updates - no tuning of the learning rate, no

annealing, no luck required. The proof bounds the number of weight updates by a constant that depends only on the geometry of the data: how far the closest point sits from the best possible boundary, relative to the size of the inputs.

That is a remarkably strong promise, and it is the reason the perceptron caused such excitement in 1958. It also sharpens the sting of the XOR result. The theorem's single hypothesis is linear separability; when that holds, convergence is certain, and when it fails - as it does for XOR - the theorem says nothing, and in fact the rule never converges, cycling through weight vectors forever. The experiment shows both faces of the same theorem: four epochs to perfection on separable data, and an unbounded, never-converging run on XOR that the test caps at two hundred epochs and checks has not converged.

ADALINE reaches the same separating boundary by a different road, and the contrast is instructive. Where the perceptron updates only on misclassified points and stops dead once everything is correct, ADALINE descends the mean-squared error continuously, adjusting the weights by the delta rule on every point whether it is classified correctly or not. On separable data both arrive at full accuracy; the difference is that ADALINE's smooth error descent generalises to the continuous, differentiable training that multi-layer networks need, while the perceptron rule does not. That is the quiet reason the delta rule, not the perceptron rule, is the ancestor of modern training.

CHAPTER 7

Design decisions worth defending

Six choices that shaped the project, each with the reason it was made and the alternative it was made against - the judgement behind the code, stated so it can be argued with.

7.1 A queue model, not a car-following one

The simulator stores and forwards vehicles as queues rather than tracking each car's position and speed. The alternative - a car-following microsimulation - is more realistic but stochastic and slow, and it would make the objective an optimiser calls thousands of times both noisy and expensive. The queue model is deterministic, fast, and gives delay an exact meaning as the integral of queue length. The cost is that vehicle dynamics - shockwaves, acceleration, lane changes - are out of scope, and the results are about timing and coordination only. That is a trade made deliberately, not an oversight.

7.2 Centroid defuzzification, over cheaper alternatives

The fuzzy controller defuzzifies by centroid. Mean-of-maxima is cheaper and bisector is simpler, but both discard information: they answer with the peak or a split point rather than the whole aggregated shape. The centroid gives every rule that fired a proportional say, which is what makes the controller respond smoothly to a queue that is partly medium and partly long. With singleton consequents it costs only a weighted average, so the extra fidelity is nearly free.

7.3 Elitism in the genetic algorithm

The GA carries its best individual unchanged into each new generation. Without elitism, crossover and mutation can lose a good solution before it is improved on, and the run's best can go backwards - which makes the comparison against grid search's guaranteed optimum unfair to the GA for the wrong reason. Elitism guarantees monotone progress at the cost of a little diversity, a trade that suits a problem where the goal is to match a known optimum cheaply rather than to explore forever.

7.4 Bipolar states in the Hopfield network

The Hopfield network uses states in $\{-1, +1\}$ rather than $\{0, 1\}$. The bipolar convention makes the Hebbian outer product symmetric around zero, so an off bit contributes as strongly as an on bit and the stored patterns sit as balanced energy minima. The 0/1 convention biases the weights and weakens recall; bipolar is the standard choice for exactly that reason, and the capacity result of $0.138 N$ is stated for it.

7.5 Pinning the first offset

Both optimisers fix intersection zero's offset at zero. Because only relative offsets affect progression, a global shift of every offset is the same solution, so the unpinned search space is larger by a whole factor of the cycle length with no new solutions in it. Pinning removes that redundant dimension, which shrinks grid search from a hopeless enumeration to a feasible one and gives the GA a smaller space to work - a free simplification that changes nothing about the answer.

7.6 Seeding everything, including the heuristic

Every random draw - arrivals, initial GA population, mutation, Hopfield patterns, neuron initialisation - comes from a seeded generator. That is what lets the reports reproduce exactly and the tests assert on numbers rather than ranges. It matters most for the genetic algorithm: a heuristic whose result changed run to run could not be compared cleanly against grid search's fixed optimum, and 'the GA matched it' would be a claim about one lucky run rather than a reproducible fact.

CHAPTER 8

How the reports are made

Every number in this book is written to reports/.json by one run of run.py. This is the code that produces it, so each figure's provenance is as inspectable as the algorithms.*

8.1 One writer, one function per experiment

Each experiment is a function that runs the methods and hands a dictionary to one writer; the book and the README read those files at build time, so a figure can always be traced to the run that produced it.

Listing R.1 - the report writer

run.py

```
def write(name: str, payload: dict) -> None:
    REPORTS.mkdir(exist_ok=True)
    (REPORTS / name).write_text(json.dumps(payload, indent=2), encoding="utf-8")
    print(f" wrote reports/{name}")
```

Listing R.2 - the fuzzy experiment, sweeping a fair baseline

run.py

```

def run_fuzzy() -> None:
    print("[fuzzy] adaptive green time versus the best-tuned fixed split")
    # Asymmetric, time-varying demand: the main street heavy first, then the
    # cross street - exactly where a fixed split wastes green on an empty road.
    demand = [(0.45, 0.08)] * 6 + [(0.08, 0.45)] * 6

    # The baseline must be the BEST fixed green, not an arbitrary one, or the
    # comparison is a strawman. Sweep the constant green and keep the best.
    fixed_sweep = {}
    best_fixed_delay = float("inf")
    best_fixed_green = None
    for g in range(10, 51, 5):
        r = simulate_isolated(FixedGreenController(g), demand, sat_flow=1.0, seed=SEED)
        fixed_sweep[g] = round(r.total_delay, 1)
        if r.total_delay < best_fixed_delay:
            best_fixed_delay = r.total_delay
            best_fixed_green = g

    fuzzy = simulate_isolated(FuzzyGreenController(), demand, sat_flow=1.0, seed=SEED)
    reduction = (best_fixed_delay - fuzzy.total_delay) / best_fixed_delay
    write("fuzzy.json", {
        "demand_profile": "main-street-heavy then cross-street-heavy",
        "fixed_green_sweep": fixed_sweep,
        "best_fixed_green": best_fixed_green,
        "best_fixed_delay": round(best_fixed_delay, 1),
        "fuzzy": fuzzy.as_dict(),
        "delay_reduction_vs_best_fixed": round(reduction, 4),
        "sample_rules": {
            "my_long_other_short": round(FuzzyGreenController().infer(16, 4), 1),
            "my_short_other_long": round(FuzzyGreenController().infer(2, 14), 1),
            "both_medium": round(FuzzyGreenController().infer(10, 10), 1),
        }
    })

```

8.2 The offset and soft-computing experiments

The offset experiment runs fixed, grid and GA on the same objective and records the evaluation saving and the scaling figure; the neuron experiment runs both learners on the separable set and on XOR.

Listing R.3 - fixed, grid and GA on one objective

run.py

```

n, cycle, step = 4, 60, 5
fixed_delay = objective([0] * n)

t = time.perf_counter()
grid = gridsearch.search(objective, n, cycle, step)
grid["seconds"] = round(time.perf_counter() - t, 2)

t = time.perf_counter()
ga = genetic.optimise(objective, n, cycle, step, seed=SEED)
ga["seconds"] = round(time.perf_counter() - t, 2)

matched = ga["best_delay"] <= grid["best_delay"] * 1.001

```

Listing R.4 - the neuron experiment

run.py

```
def run_neurons() -> None:
    print("[neurons] perceptron and ADALINE, and the XOR wall")
    Xsep, ysep = linearly_separable(SEED)
    perc = Perceptron(2, seed=SEED).fit(Xsep, ysep)
    ada = Adaline(2, seed=SEED).fit(Xsep, ysep)

    Xxor, yxor = xor_problem()
    perc_xor = Perceptron(2, seed=SEED).fit(Xxor, yxor, epochs=200)
    ada_xor = Adaline(2, seed=SEED).fit(Xxor, yxor, epochs=500)

    write("neurons.json", {
        "linearly_separable": {
            "perceptron": {k: perc[k] for k in ("converged_epoch", "final_accuracy")},
            "adaline": {k: ada[k] for k in ("final_mse", "final_accuracy")},
        },
        "xor": {
            "perceptron_accuracy": perc_xor["final_accuracy"],
            "perceptron_converged": perc_xor["converged_epoch"] is not None,
            "adaline_accuracy": ada_xor["final_accuracy"],
            "note": "Neither single-layer unit can solve XOR - no line separates it.",
        }
    })
```

8.3 Hopfield, and the whole run

The Hopfield experiment produces both curves and locates the measured capacity cliff; the top-level main runs the four experiments in order to produce the complete reports directory this book is built from.

Listing R.5 - the Hopfield experiment

run.py

```
def run_hopfield() -> None:
    print("[hopfield] recovering corrupted detector readings")
    size = 64
    recovery = recovery_curve(size, n_patterns=3, seed=SEED, trials=200)
    capacity = capacity_curve(size, seed=SEED, trials=200)
    # the theoretical capacity is about 0.138 N; find where recovery drops below
    # 0.9 in the measured curve
    cliff = next((row["load_ratio"] for row in capacity
                  if row["recovery_rate"] < 0.9), None)
    write("hopfield.json", {
        "network_size": size,
        "theoretical_capacity_ratio": 0.138,
        "recovery_curve": recovery,
        "capacity_curve": capacity,
        "measured_capacity_cliff_ratio": cliff,
    })
```

Listing R.6 - running every experiment in order

run.py

```
def main() -> int:
    run_fuzzy()
    run_offsets()
    run_hopfield()
    run_neurons()
    print("all reports written")
```

8.4 Why seeds, and why that is not enough

Every number in this book is read from a JSON file written by a real run, never typed into the prose. That rule is the difference between a measurement and a claim, and it is enforced mechanically: the build pulls each figure from reports/*.json by key, so if a run changed a result the book would change with it or fail to build. There is no path by which a stale or invented number reaches the page.

Reproducibility rests on a single seed - 20260911 - threaded through every stochastic step: the demand profiles, the genetic algorithm's selection and mutation, the Hopfield corruption trials, the neurons' weight initialisation. Because the traffic model itself is a deterministic queue simulation, fixing the seed fixes every number exactly, on any machine, run to run. The genetic algorithm's path is identical too, so even the heuristic's convergence generation reproduces rather than merely its endpoint.

But a seed is a weaker promise than it first appears, and the book is careful about what it claims. A seed guarantees that *this code* produces *these numbers* again; it does not guarantee the numbers are right, and it does not survive a change in NumPy's random implementation or a refactor that draws random numbers in a different order. The robust claims are therefore the *directions* - fuzzy beats the best fixed split, the GA matches the grid far cheaper, recovery collapses past capacity, XOR defeats a single layer - which hold across seeds because they are properties of the methods. The exact percentages are seed-dependent artefacts of the particular demand profiles, and the reports record those profiles in full so the reader can see precisely what was measured rather than taking the headline on trust.

CHAPTER 9

Results, and what this does not establish

Every claim, where to check it, and the limits stated plainly.

9.1 What was shown

Table 7.1 - every claim, and the file that backs it

Claim	Evidence
Fuzzy control beats the best-tuned fixed split	reports/fuzzy.json
The GA matches grid search's optimum in far fewer simulations	reports/offsets.json
The GA's advantage scales with network size	reports/offsets.json
Hopfield recovers corrupted patterns up to its capacity	reports/hopfield.json
Perceptron and ADALINE converge on separable data	reports/neurons.json
Neither single-layer unit can solve XOR	reports/neurons.json

9.2 What it does not establish

- **The traffic model is store-and-forward, not car-following.** It captures delay, progression and green fragmentation; it does not model lane changes, acceleration, or turning movements.
- **The GA ties grid search on quality; it wins on time.** On a small network grid search is perfectly usable. The GA's value is that it keeps the quality affordable as the exhaustive search explodes with size.
- **The soft-computing components are single, from-scratch implementations.** Legibility was chosen over the tuned, vectorised versions a production system would use.
- **Results are for the seeded demand profiles shown.** The direction of each finding is robust; the exact percentages depend on the demand asymmetry and the seed.
- **A green wave is a one-way idealisation.** Real arterials carry two directions whose progression bands conflict, a harder problem this model does not take on.

THE ONE CLAIM WORTH DEFENDING

Not that any method is best in general - the fuzzy controller and the GA each win a specific, stated comparison, and the neurons hit a wall. The claim is that each was measured against a fair baseline and reported honestly, including where the clever method only tied. A heuristic praised without an exhaustive control to check it against is a story; these have the control.

CHAPTER 10

How to read these results

Four methods, four honest baselines, and no overall winner.

10.1 Each win is a specific, bounded claim

It would be easy to read this project as "soft computing beats the classics", and that would be the wrong lesson. The fuzzy controller beats the best fixed split *on demand that shifts*; on steady, symmetric demand a fixed split is fine and the fuzzy system only matches it. The genetic algorithm beats grid search *on time, and only as the network grows*; on the small arterial grid search is perfectly usable and the two tie on quality. The Hopfield network recovers detectors *up to a capacity* it visibly falls off. The perceptron learns *separable* data and hits a wall on XOR. Every result is a bounded claim with a regime attached, and stripping the regime off to make a slogan would make each one false.

10.2 The baseline is the point

What ties the four together is not the methods but the discipline: each is measured against the fairest baseline available, and where it only ties, the report says tie. The best-tuned fixed split, not an arbitrary one. Exhaustive grid search, not a strawman heuristic. The stored patterns themselves as ground truth. A second learning rule as a foil. A method that looks good only against a weak baseline has proven nothing, and a large improvement over a poorly chosen comparison is less trustworthy, not more. The numbers here are smaller for being honest, and worth more for it.

10.3 When to reach for each

The practical reading is a decision guide. Reach for a fuzzy controller when an expert can state the policy in vague rules but no clean model of the plant exists - it turns "if the queue is long, hold green" directly into control. Reach for a genetic algorithm when the search space is too large to enumerate and solutions combine meaningfully, so crossover can build good wholes from good parts. Reach for an associative memory when inputs are noisy and the valid states are few and known. And reach past a single linear unit the moment the classes are not separable - which, for anything interesting, is almost always.

CHAPTER 11

Exercises

With worked solutions. The last has no clean answer on purpose.

11.1 Questions

1. The fuzzy membership sets overlap - a queue of nine is partly medium and partly long. Why is that overlap essential rather than sloppy, and what would sharp, non-overlapping sets cost?
2. The genetic algorithm ties grid search on quality. In what sense, then, does it win, and why is that win invisible on a small network and decisive on a large one?
3. Only relative offsets matter, so the first intersection is pinned at zero. Why does that pinning not lose any solutions, and what does it save?
4. A Hopfield network of 64 neurons stores maybe nine patterns reliably. Where does the 0.138 N limit come from intuitively, and what happens to recall past it?
5. The perceptron converges on the separable data but never on XOR. State precisely what property of XOR defeats it, and why ADALINE - a different learning rule - fails at the same wall.
6. The fuzzy controller is compared against the best fixed split, found by a sweep. Why would comparing against an arbitrary fixed green have been a weaker result even if the percentage looked larger?
7. The traffic model is store-and-forward (queues), not car-following (individual vehicles with acceleration). What does that choice buy for this project, and what class of question does it put out of reach?
8. The genetic algorithm caches every offset vector it simulates. Why does that matter specifically for the comparison against grid search, as opposed to merely being an optimisation?
9. Fuzzy membership grades and probabilities both live in $[0, 1]$. In what sense is a queue being 0.7 'long' NOT a 70% chance of anything, and why does the distinction matter for the controller?
10. A hidden layer lets a network solve XOR that a single layer cannot. Intuitively, what does the hidden layer add that a straight boundary lacks?
11. The Hopfield network sometimes settles into a pattern that was never stored. What are these spurious states, and why does adding more stored patterns make them worse?
12. Grid search here is feasible only because the offset step is coarse and the arterial short. If you halved the step to get finer offsets, what happens to grid search's cost and to the GA's, and which choice would you make?

11.2 Solutions

1.

The overlap is what makes the output continuous. Because a queue of nine belongs partly to medium and partly to long, the green time it produces is a blend of the two rules' outputs, and it shifts smoothly as the queue grows. Sharp sets would make the controller jump discontinuously at the boundary - eight vehicles one green, nine a very different one - which is both unphysical and unstable, oscillating as the queue hovers at the cutoff. The overlap is the whole reason fuzzy control is smooth.

2.

It wins on time-to-solution: it reaches the same optimum in far fewer simulations. On a small network the exhaustive grid is cheap, so saving simulations saves little absolute time and the win is invisible. As the network grows the grid is $(\text{cycle/step})^{(n-1)}$ - exponential in the number of intersections - while the GA's cost grows far more slowly, so the same-quality solution that grid search can no longer afford is still cheap for the GA. The win is asymptotic, not per-instance.

3.

Shifting every offset by the same constant shifts when all the signals are green together, but not their relationship to each other - and progression depends only on the relationships. So the family of offset vectors that differ by a global shift are equivalent, and pinning the first to zero picks one representative from each family without discarding any distinct solution. It divides the search space by the cycle length - one whole dimension removed.

4.

Each stored pattern adds a term to every weight, and the recall of one pattern sees the others as noise on its own signal. As more patterns are stored, that cross-talk grows until it can flip a neuron away from the pattern being recalled; the 0.138 N figure is where, for random patterns, the noise typically overwhelms the signal. Past it, the stored patterns stop being stable fixed points - recall lands in spurious mixtures instead - which the capacity table shows as the collapse from near-1 to near-0.

5.

XOR is not linearly separable: no single straight line puts its two positive points on one side and its two negatives on the other. The perceptron can only represent a linear boundary, so there is no weight vector that classifies XOR, and its rule therefore never reaches zero errors - it cycles forever. ADALINE uses a different rule, minimising squared error rather than counting mistakes, but it is still a single linear unit drawing a single straight boundary, so it hits the identical wall: a better optimiser of the wrong hypothesis class cannot escape the class.

6.

Because an arbitrary fixed green might just be badly chosen, and then the fuzzy controller's win would partly measure the baseline's incompetence rather than its own skill. Sweeping the fixed green and comparing against the best one removes that confound: whatever margin remains is genuinely due to adapting to demand, not to out-timing a strawman. A larger percentage over a weak baseline is a worse result, because it is less trustworthy - honesty of the comparison beats size of the number.

7.

Store-and-forward buys determinism, speed, and a clean definition of delay as the integral of queue length - which is exactly what an optimiser needs as a fast, repeatable objective it can call thousands of times. It puts out of reach anything that depends on individual vehicle dynamics: shockwaves propagating back through a queue, the effect of acceleration on saturation flow, lane changing, and turning conflicts. Those need a car-following microsimulation, which is far slower and stochastic - the wrong objective for a search, and the honest limit stated in Chapter 7.

8.

Because the comparison's whole currency is number of simulations, and without the cache the two methods would be counting different things. Grid search evaluates each grid point once by construction. A genetic algorithm, left uncached, re-simulates chromosomes it has seen before - elitism alone re-visits the best every generation - so its raw call count would overstate its true cost. Caching makes the GA's reported evaluations distinct simulations, the same unit grid search is measured in, so the 90% saving is a like-for-like claim rather than an artefact of how the loop happens to be written.

9.

A probability of 0.7 says a proposition is definitely true or false and you are uncertain which; repeated trials would resolve it. A membership of 0.7 says the queue is, right now and with certainty, partly long - there is nothing to resolve, no trial to run, and it will still be 0.7 long tomorrow. Fuzziness is vagueness of the predicate, not uncertainty about a fact. It matters for the controller because the grades are combined with min and max and defuzzified by centroid - operations that make sense for degrees of truth but would be wrong for probabilities, which would call for multiplication and normalisation instead.

10.

A hidden layer lets the network draw more than one line and then combine the regions. XOR needs the two diagonal corners separated from the other two, which no single straight cut achieves; but two lines carve the plane into regions that a second layer can OR together into the right answer. The hidden units learn intermediate features - effectively 'top-left-ish' and 'bottom-right-ish' - and the output combines them. The single layer has no such intermediate representation, only the raw inputs and one boundary.

11.

Spurious states are stable patterns the network settles into that were never deliberately stored - typically mixtures or negatives of the stored patterns, which the Hebbian rule accidentally also makes into energy minima. They are the associative-memory analogue of a false memory. Adding more stored patterns multiplies the cross-terms in the weight matrix, which both weakens the genuine minima and creates more of these accidental ones, so recall is increasingly likely to land in a spurious basin - the mechanism behind the capacity cliff.

12.

Halving the offset step doubles the resolution per intersection, so grid search's cost, which is (cycle/step) to the power of the free intersections, rises by a factor of two per intersection - for the four-intersection arterial, roughly eight times the simulations. The GA's cost barely moves: a finer grid is a slightly larger alphabet for the same chromosome, and it still converges in a few hundred evaluations. If I needed the finer offsets I would reach for the GA precisely because grid search's cost is exponential in exactly the parameter I just refined.

CHAPTER 12

Running the Project, Step by Step

Every terminal command needed to go from a fresh clone to a finished run, what each one does, and what to do when one of them fails.

12.1 Prerequisites

You need Python 3.11 or newer and Git. Verify both before starting - most reported 'the project does not work' turns out to be an environment that was never activated.

Check your toolchain

```
python --version
git --version
```

12.2 Step 1 - Clone and enter the repository

Get the code

```
git clone https://github.com/Nikhil201716/Cadence-Traffic-Control.git
cd Cadence-Traffic-Control
```

12.3 Step 2 - Create an isolated environment

A virtual environment keeps this project's dependencies from colliding with anything else on your machine.

Create and activate

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

IF POWERSHELL REFUSES TO RUN THE ACTIVATION SCRIPT

Windows blocks unsigned scripts by default. Allow them for your own user account only:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

12.4 Step 3 - Install dependencies

Install

```
pip install -r requirements.txt
```

12.5 Step 4 - Run the pipeline, stage by stage

Each command below produces the input the next one consumes. Run them in this order:

The full pipeline in order

```
pip install -r requirements.txt
python tests/test_trafficsim.py
python tests/test_fuzzy.py
python tests/test_optimisers.py
python tests/test_softcomputing.py
python run.py
```

What each stage does:

1. `requirements.txt` - NumPy, the only dependency
2. `tests/test_trafficsim.py` - the simulator and the fuzzy comparison
3. `tests/test_fuzzy.py` - membership functions and inference
4. `tests/test_optimisers.py` - the GA-versus-grid-search comparison
5. `tests/test_softcomputing.py` - Hopfield capacity and the XOR wall
6. `run.py` - every experiment; writes reports/*.json

EXPECTED OUTPUT

Python 3.11 or newer and NumPy - one pip install, and the plan allows no more. Every number in this book is read from reports/*.json, so run.py must run before build_book.py. Everything is seeded, including the genetic algorithm, so the numbers reproduce exactly.

12.6 Command reference card

Every command from this chapter in one block, for when you already know what you want.

Complete command reference

```

# --- setup (once) ---
git clone https://github.com/Nikhil201716/Cadence-Traffic-Control.git
cd Cadence-Traffic-Control
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows
source .venv/bin/activate           # macOS / Linux
pip install -r requirements.txt

# --- run ---
pip install -r requirements.txt
python tests/test_trafficsim.py
python tests/test_fuzzy.py
python tests/test_optimisers.py
python tests/test_softcomputing.py
python run.py

# --- finish ---
deactivate

```

12.7 Troubleshooting

Common problems and their fixes

Symptom	Cause	Fix
<code>ModuleNotFoundError</code>	environment not activated	re-run the activate command from Step 2
<code>FileNotFoundError</code> on a data file	an earlier stage was skipped	run the stages in the documented order
Numbers differ from this book	a seed or parameter was changed	check the constants at the top of the generator
command not found	dependency missing from this env	<code>pip install -r requirements.txt</code> with the env active
Permission denied writing output	directory created by another user or process	delete the generated output directory and re-run

CHAPTER 13

Appendix

Repository map, glossary, and reproducibility notes.

13.1 A - Repository map

Where everything lives

Path	Purpose
src/cadence/trafficsim.py	the store-and-forward simulator (arterial and isolated)
src/cadence/fuzzy.py	the Mamdani fuzzy green-time controller
src/cadence/gridsearch.py	exhaustive offset search - the honest baseline
src/cadence/genetic.py	the genetic algorithm for offsets
src/cadence/hopfield.py	the associative memory for detector recovery
src/cadence/neurons.py	the perceptron and ADALINE
tests/	zero-dependency correctness tests for every component
run.py	runs every experiment and writes reports/*.json
reports/	the evidence behind every number in this book

13.2 B - Glossary

Terms used in this book

Term	Meaning
Seeded generator	a synthetic data producer whose random draws are reproducible from a fixed starting value
Ground truth	the mechanism deliberately injected into synthetic data, used only to score an analysis - never to make a decision
Control arm / baseline	a simple alternative a new method must beat before it is worth its complexity
Point-in-time correctness	computing a feature using only information that existed at the moment being modelled
Leakage	information from the future or from the target entering a feature, inflating every offline metric
Reproducibility checksum	a hash of generated output, compared across a fresh process to prove determinism
Store-and-forward model	A queue model of traffic where delay is the integral of queue length over time.
Green wave / progression	Offsetting signals so a platoon meets green at each one in turn.
Offset	The start of a signal's green within the common cycle, relative to a reference.
Mamdani FIS	A fuzzy inference system: fuzzify inputs, fire min-implication rules, defuzzify by centroid.
Membership function	How strongly a value belongs to a fuzzy set, between 0 and 1.
Defuzzification	Turning an aggregated fuzzy output back into one crisp number.
Genetic algorithm	A population search using selection, crossover and mutation.
Elitism	Carrying the best individual unchanged into the next generation.
Hopfield network	A recurrent associative memory storing patterns as attractors.
Hebbian rule	Weights set by the outer product of stored patterns.
Capacity (0.138 N)	The number of random patterns a Hopfield net of N neurons stores reliably.
Perceptron	A single linear unit trained by the sign of its output; converges iff separable.
ADALINE	A single linear unit trained by the delta (LMS) rule on its continuous output.
Linear separability	Whether one straight boundary can split the classes; XOR is not separable.

13.3 C - Reproducibility

Every dataset in this project is produced by a seeded generator, so the whole pipeline can be rebuilt from nothing and should produce byte-identical output.

A FIXED SEED IS A CLAIM, NOT A GUARANTEE

Two projects in this portfolio shipped generators with fixed seeds that were still not reproducible across runs. The causes were `rng.choice(list(a_set))` and `abs(hash(some_string))` - both of which are perfectly stable *within* one process and vary between processes, because Python randomises string hashing per interpreter run.

Neither is visible from reading the code, from the seed, or from the output looking sensible. Both were caught only by hashing the generated files and comparing across a *fresh* process. The defences used throughout this portfolio are to sort any collection before it feeds a random draw, and to verify by checksum in a new process rather than trusting the seed.

13.4 D - Where this leads next

The clearest extensions follow the limits in Chapter 7: a two-way arterial, where the progression bands in each direction conflict and no single offset satisfies both; a multi-layer perceptron, which crosses the XOR wall the single layer cannot; and a car-following microsimulation to check the store-and-forward delays against something with acceleration and lane changes. On the optimisation side, comparing the genetic algorithm against simulated annealing would show whether the population is pulling its weight or a simpler local search would do.